

UNIVERSIDAD DE EL SALVADOR

Facultad Multidisciplinaria De Occidente
Departamento de Ingeniería y Arquitectura

Ingeniería En Desarrollo De Software/Educación En Línea
LOGICA DE PROGRAMACION
SEMESTRE I / 2025



ACTIVIDAD: GUÍA DE LABORATORIO NÚMERO 9

Sobrecarga de Operadores de extracción e inserción, archivos orientados a objetos, flujos y Archivos de texto.

Nombre: _____, Carnet: _____

Fecha: _____, Grupo teórico: _____

Objetivo: Conocer los conceptos y el uso de la sobrecarga de operadores de extracción e inserción, archivos orientados a objetos, flujos y archivos de texto además de exponer una breve introducción a la sobrecarga en C++.

Indicación general: Leer detenidamente cada una de las tres partes en que está dividida la actividad y brindar respuestas en las que se solicite.

Parte 1 – Sobrecarga de Operadores en C++

Indicación: leer detenidamente el contenido de sobrecarga de operadores, en el lenguaje de programación C++.

Los operadores de C++, al igual que las funciones, pueden ser sobrecargados (overloaded). Este es uno de los aspectos más característicos de este lenguaje. La sobrecarga de operadores quiere decir que se pueden redefinir algunos de los operadores existentes en C++ para que actúen de una determinada manera, definida por el programador, con los objetos de una clase determinada. Esto puede ser muy útil, por ejemplo, para definir operaciones matemáticas con elementos tales como vectores y matrices. Así, sobrecargando

adecuadamente los operadores suma (+) y asignación (=), se puede llegar a sumar dos matrices con una sentencia tan simple como: **C=A+B;**

Otra capacidad muy utilizada es la de sobrecargar los operadores de inserción y extracción en los flujos de entrada y salida (>> y <<), de manera que puedan imprimir o leer estructuras o clases complejas con una sentencia estándar.

Los únicos operadores de C que no se pueden sobrecargar son el operador punto (.), el if aritmético (?:) y el operador **sizeof**. C++ añade otros 2 a esta lista: el scope resolution operator (::) y puntero a miembro de un objeto (*).

El objetivo último de la sobrecarga de operadores es simplificar al máximo el código a escribir, a cambio de complicar algo la definición de las clases. Una clase que disponga de operadores sobrecargados es una clase más compleja de definir, pero más sencilla e intuitiva de utilizar.

Las ventajas de la sobrecarga de operadores terminan cuando se utiliza de modo que añade complejidad o confusión a los programas. Por ejemplo, aunque esté permitido por el lenguaje, no se deberá nunca utilizar el operador (-) para multiplicar matrices o el (+) para imprimir vectores.

La sobrecarga de operadores tiene algunas limitaciones teóricas y una práctica:

- Se puede modificar la definición de un operador, pero no su gramática, es decir, el número de operandos sobre los que actúa, la precedencia y la asociatividad.
- Es necesario que al menos un operando sea un objeto de la clase en la que se ha definido el operador sobrecargado.
- La sobrecarga de operadores puede resultar bastante ineficaz, desde el punto de vista de utilización de los recursos del ordenador.

Un operador puede estar sobrecargado o redefinido varias veces, de tal manera que actúe de un modo distinto dependiendo del tipo de objetos que tenga como operandos. Es precisamente el tipo de los operandos lo que determina qué operador se utiliza en cada caso.

Un operador sobrecargado puede ser miembro o **friend** de la clase para la que se define (nunca las dos cosas a la vez). El que se defina de una forma u otra es en ocasiones cuestión

de conveniencia o incluso de preferencia personal, mientras que en otros casos la decisión está impuesta. Habitualmente:

Se suelen declarar miembros de la clase los operadores unarios (es decir, aquellos que actúan sobre un único objeto), o los que modifican el primer operando, como sucede con los operadores de asignación. Por el contrario, los operadores que actúan sobre varios objetos sin modificarlos (por ejemplo, los operadores aritméticos y lógicos) se suelen declarar como **friends**.

Para los operadores sobrecargados miembro de una clase, el primer operando debe de ser siempre un objeto de esa clase, en concreto el objeto que constituye el argumento implícito. En la declaración y en la definición sólo hará falta incluir en la lista de argumentos el segundo operando. En los operadores **friend** el número de argumentos deberá ser el estándar del operador (unario o binario).

Parte 2 – Precedencia y asociatividad de operadores en C++

Indicación: leer detenidamente el contenido de precedencia y asociatividad de operadores, en el lenguaje de programación C++.

En matemáticas, una operación es un cálculo matemático que involucra cero o más valores de entrada (llamados operandos) que produce un nuevo valor (llamado valor de salida). La operación específica que se va a realizar se denota mediante una construcción (normalmente un símbolo o un par de símbolos) denominada operador.

Por ejemplo, de niños todos aprendemos que $2 + 3$ es igual a **5**. En este caso, los literales 2 y 3 son los operandos, y el símbolo + es el operador que nos dice que apliquemos sumas matemáticas a los operandos para producir el nuevo valor. 5.

Precedencia del operador

Consideremos una expresión más complicada, como $4 + 2 * 3$. Una expresión que tiene múltiples operadores se llama expresión compuesta. Para evaluar esta expresión compuesta, debemos comprender tanto lo que hacen los operadores como el orden correcto para aplicarlos. El orden en que se evalúan los operadores en una expresión compuesta está determinado por la precedencia de un operador. Usando las reglas de precedencia matemática normales (que establecen que la multiplicación se resuelve antes que la suma), sabemos que la expresión anterior debe evaluarse como $4 + (2 * 3)$ para producir el valor 10.

En C++, cuando el compilador encuentra una expresión, debe analizarla de manera similar y determinar cómo debe evaluarse. Para ayudar con esto, a todos los operadores se les asigna un nivel de precedencia. Primero se evalúan los operadores con el nivel más alto de precedencia.

Asociatividad de operadores

¿Qué sucede si dos operadores en la misma expresión tienen el mismo nivel de precedencia? Por ejemplo, en la expresión $3 * 4 / 2$, los operadores de multiplicación y división tienen nivel de precedencia 5. En este caso, el compilador no puede depender únicamente de la precedencia para determinar cómo evaluar el resultado.

Si dos operadores con el mismo nivel de precedencia son adyacentes en una expresión, la asociatividad del operador le dice al compilador si debe evaluar los operadores de izquierda a derecha o de derecha a izquierda. Los operadores en el nivel de precedencia 5 tienen una asociatividad de izquierda a derecha, por lo que la expresión se resuelve de izquierda a derecha: $(3 * 4) / 2 = 6$.

Uso de paréntesis

En aritmética normal, se aprendió que se puede usar paréntesis para cambiar el orden de aplicación de las operaciones. Por ejemplo, sabemos que $4 + 2 * 3$ se evalúa como $4 + (2 * 3)$, pero si desea que se evalúe como $(4 + 2) * 3$, puede poner explícitamente entre paréntesis la expresión para que se evalúe de la manera que uno quiere. Esto funciona en C++ porque los paréntesis tienen uno de los niveles de precedencia más altos, por lo que los paréntesis generalmente se evalúan antes que lo que hay dentro de ellos.

Para reducir los errores y hacer que su código sea más fácil de entender sin hacer referencia a una tabla de precedencia, es una buena idea poner entre paréntesis cualquier expresión compuesta no trivial, para que quede claro cuál es su intención.

Parte 3- Desarrollar los siguientes puntos, ponderación de actividad 100%

Indicación: Brindar solución a los siguientes puntos y desarrolla software si es parte de lo solicitado, tomando como referencia los tópicos abordados en clase

1. Coloque en comentarios la definición de "Sobrecarga de Funciones" dentro del ej. siguiente.
2. Escriba un programa que calcule el área de distintas figuras geométricas (triángulo, cuadrado, rectángulo y círculo), sobrecargando la función área con el número y tipo de argumentos necesarios para cada tipo de figura. (se debe hacer a través de código en C++ o pseudocódigo). Deberá utilizar la "Sobrecarga de Funciones"

NOTA: Las indicaciones de envío de esta actividad, estarán junto a este documento.